

LLM 后训练之 Evaluation

从 Benchmark 到持续评测系统：能力、行为、Agent、产品与 Failure-Driven Post-Training

Post-Training Research Handbook · 2026-08

Know-Why × Know-How × Eval Contract × Failure Flywheel

执行摘要：Evaluation 不是训练后的“验收”，而是后训练的控制系统

在现代 LLM 后训练中，Evaluation 不应被放在流水线末尾。它同时承担三种职责：定义“什么叫进步”、检测“哪里退化”、把失败编译成下一轮数据与奖励信号。因此，一个成熟 Eval System 实际上是 Objective、Data、Reward、Training 与 Product 之间的反馈控制器。

核心定义

Evaluation = Specification × Task Distribution × Grading × Statistical Reliability × Slicing × Regression × Failure Mining。Benchmark 只是 Eval System 的一个输入，不是 Evaluation 本身。

层	核心问题	主要产物
Eval Objective	究竟要证明什么？	Capability/Behavior/Product Eval Contract
Task Set	在哪些任务、难度和分布上测？	Benchmark + Private/Online Regression Set
Grader	什么叫成功？	Exact Match / Test / Rule / Human / LLM Judge
Statistics	这个差异可靠吗？	CI、显著性、方差、重复运行
Slicing	平均分掩盖了哪些失败？	Domain/Difficulty/Length/Language/Risk slices
Regression	新模型改进是否破坏旧能力？	Gating + Release Criteria
Feedback	失败如何进入下一轮训练？	Failure Taxonomy → Data/Reward/Prompt Queue

OpenAI 将 evals 概括为“Specify → Measure → Improve”；Anthropic 在 2026 年的 agent eval 工程总结中同样强调，agent 的多轮工具使用、状态修改和适应性让传统静态 benchmark 不再足够，必须组合不同 grader 和环境级结果。[12][13]

1. 第一性原理：Evaluation 到底在测什么？

1.1 Model Capability、System Capability 与 Product Utility 必须分开

层级	对象	例子	为什么不能混
Model Eval	裸模型/统一 prompt	MATH、coding、instruction following	便于比较模型，但忽略 scaffold/tool/context
System Eval	模型 + prompt + RAG + tool + agent runtime	SWE-bench agent、WebArena	真实能力来自系统组合
Product Eval	完整产品 + 用户流程 + 延迟/成本	客服解决率、人工接管率	最终用户价值可能与 benchmark 排名不同

同一 base model 在不同 scaffold、工具集、上下文与搜索预算下，结果可能显著不同。对 Agent 更是如此：评测单位逐渐从“回答文本”变成“环境中的完整轨迹与最终状态”。

1.2 Evaluation 是 Objective 的可执行版本

```
Objective Spec
└ compile
Eval Contract
├ task distribution
├ grader / oracle
├ metrics
├ slices
├ confidence rules
└ release gates
```

Know-Why

如果 Objective 说“可靠完成复杂软件任务”，而 Eval 只测静态代码补全，那么训练很可能优化错方向。Eval 的第一责任是让目标可观测。

2. 从 Benchmark 到 Eval Portfolio：为什么单一分数会误导？

HELM 的核心贡献之一，是把语言模型评测从“少数 accuracy benchmark”扩展为 scenario × metric 的多维评测：accuracy、calibration、robustness、fairness、bias、toxicity、efficiency 等同时出现，暴露 trade-off。[1]

单 Benchmark 思维	Eval Portfolio 思维
一个总分	多个能力与行为切片
固定公开测试集	公开 benchmark + 私有回归集 + 动态任务
Accuracy	Correctness + Reliability + Cost + Safety + UX
一次运行	重复、置信区间、版本跟踪
排行榜	Failure discovery + Release gate

3. Evaluation Taxonomy：一套可复用的评测空间

轴	典型取值	核心问题
对象	Model / System / Agent / Product	测的是谁？
交互	Single-turn / Multi-turn / Interactive	是否存在状态与环境反馈？
答案空间	Closed / Structured / Open-ended	能否自动判分？
Ground Truth	Exact / Executable / Rubric / Human	真值来自哪里？
时间	Static / Refreshing / Live	是否容易污染和饱和？
风险	Routine / High-impact / Adversarial	最坏情况有多重要？
预算	1-shot / pass@k / search / agent budget	测试时计算是否公平？
可见性	Public / Hidden / Private	是否用于训练或调参？

3.1 Capability Eval

- 知识：事实性、领域知识、时效性。
- 推理：数学、逻辑、规划、组合泛化。
- 代码：生成、修复、repo-level engineering、测试。
- 指令遵循：格式、约束、复杂 instruction hierarchy。
- 长上下文：检索、全局综合、多跳与 full-context reasoning。

3.2 Behavior / Alignment Eval

- Helpfulness、truthfulness、uncertainty calibration。
- Policy compliance、拒答边界、越权与提示注入鲁棒性。
- Sycophancy、verbosity、style、deference、hidden-behavior audit。

3.3 Agent Eval

- Task success、trajectory quality、tool selection、state changes、recovery、cost。
- 环境是否可复现、grader 是否真正检查最终状态，而不是只看 final answer。

4. Grader / Oracle 选择：越客观越靠前

Grader	信号质量	成本	最适合
Exact match / parser	高	低	数学终值、JSON、分类
Unit test / compiler	高	中	代码
Environment state check	高	中-高	Agent、web/OS task
Rule / schema	高但覆盖窄	低	格式与约束
Reference-based scoring	中-高	低-中	摘要/抽取
LLM Judge + rubric	中	中	开放式质量、研究/写作
Human expert	高但有主观性	高	高影响开放任务

原则

能由 execution/environment verifier 判定的任务，尽量不要只用 LLM Judge；必须 Judge 的任务，则应建立 judge 自身的 calibration 与 benchmark。PaperBench 甚至单独建立 judge benchmark 来验证自动评分器。[14]

5. 自动评测指标：不仅是 Accuracy

5.1 Closed-form correctness

指标	定义/直觉	注意
Accuracy	正确样本占比	类别不平衡时不足
Exact Match	规范化后完全一致	对等价表达敏感
F1	precision/recall 调和平均	适合抽取/集合
Execution Pass	运行/测试是否通过	要防 flaky tests 与环境差异
Task Success	最终状态满足目标	Agent 最关键 outcome 指标

5.2 pass@k 与 sampling budget

代码与推理常用 pass@k 评估多次采样后的成功概率。若 n 个样本中有 c 个正确，经典无偏估计形式为：

pass@k = 1 - C(n-c, k) / C(n, k)

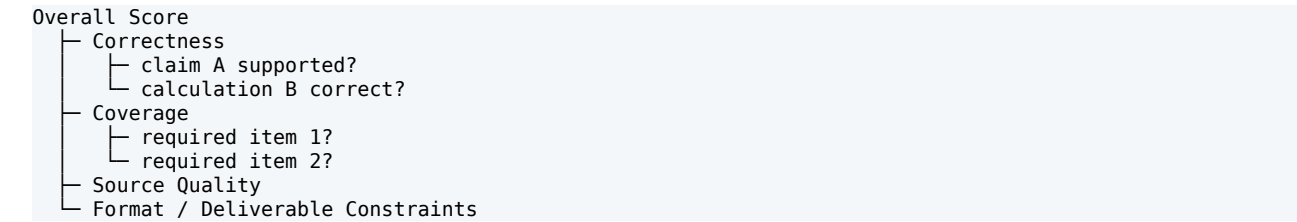
关键是必须同时报告 k、temperature、max tokens、工具预算等。否则“模型能力”和“测试时计算”被混在一个分数里。

5.3 Calibration

- Expected Calibration Error (ECE)：模型置信度是否与真实正确率匹配。
- Brier Score / Log Loss：概率预测质量。
- Selective accuracy：只在高置信度样本上回答时的准确率与覆盖率。

6. Open-ended Eval：Rubric 比“给 1-10 分”更重要

开放式任务最常见的失败是让 Judge 直接给总分。更稳定的做法是先把目标分解成可判定 proposition，再汇总。OpenAI 的 PaperBench 采用层级 rubric，把 20 篇论文复现实验拆成 8,316 个可单独评分子任务；GDPval 也强调工作产物级 rubric 与专业人员基准。[14][15]



7. LLM-as-a-Judge：什么时候能用，什么时候危险？

风险	表现	缓解
Position bias	偏好 A/B 的位置	swap test、randomize order
Verbosity bias	长回答更易获高分	长度分层、rubric 原子化
Self-enhancement	偏好自身风格/家族	异构 judge、human audit
Reference leakage	judge 被 reference 措辞牵引	blind grading、objective verifier
Reasoning failure	复杂 math/code 判错	执行/PRM/stronger judge
Scale ceiling	generator 变强后 judge 区分力下降	JudgeBench、judge calibration

JudgeBench 专门构造知识、推理、数学与代码的困难 response pairs，指出许多强 judge 在高难度客观正确性判断上仍面临明显挑战。[10] 因此 Judge 不是“免费 ground truth”。

7.1 Judge Calibration Suite

- 与专家或 objective verifier 的 pairwise agreement。
- A/B swap consistency。
- 长度、风格、模型身份、引用格式等 counterfactual tests。
- OOD difficulty slices。
- Judge refresh：当 generator 超过 judge 能力时升级。

8. Benchmark Contamination：公开 Benchmark 为什么会过期？

Benchmark contamination 会让测试数据直接或间接进入预训练、SFT、synthetic data 或 prompt tuning，从而使分数不再代表泛化。LiveBench 的设计针对这一问题：频繁更新近期来源的问题，并尽量使用可自动评分的客观 ground truth，减少公开静态测试集和 LLM judge 的双重问题。[2]

策略	作用	局限
Private holdout	降低直接泄漏	难以公共复现
Temporal refresh	训练后产生新任务	维护成本高
Dynamic generation	持续生成新样本	生成器也会偏
Canary / watermark	检测已知泄漏	覆盖有限
Similarity scan	发现近重复	语义改写仍难
Hidden tests	防针对测试作弊	需要可靠基础设施

Know-How

“decontaminated” 不是一个二元标签，而是一组证据：时间、来源、n-gram/embedding 相似度、公开程度、训练数据 provenance 与 hidden split。

9. Saturation 与 Dynamic Evaluation：Benchmark 变简单之后怎么办？

一个 benchmark 一旦接近饱和，就难以区分模型。解决方向包括：提高难度、加入 adversarial variants、扩大真实任务分布、使用最新数据、从 single-turn 迁移到交互式环境。LiveBench 的月度更新、GDPval 的真实职业任务、SWE-Lancer 的真实 freelance 工程任务，以及 Bloom 的自动行为场景生成，都体现了“评测持续生成化”。[2][15][16][17]

9.1 Eval 生命周期

Design → Pilot → Calibrate → Freeze/Hide → Run → Error Analysis → Refresh/Retire

成熟组织应当给 benchmark 本身建立状态：draft、calibrated、release-gate、saturated、contaminated、retired。

10. Long-Context Eval：NIAH 为什么不够？

HELMET 对 51 个长上下文模型的研究指出，needle-in-a-haystack 等 synthetic retrieval task 对真实下游应用预测力有限；不同长上下文类别之间相关性也很低。很多模型 NIAH 接近满分，但在 full-context reasoning 或复杂 instruction following 上差距依然明显。[3]

类别	真正要测什么
Retrieval	能否找到正确证据
Multi-hop	能否连接远距离证据
Full-context reasoning	必须综合大部分上下文
Long-document QA	引用与归因
Many-shot / in-context	能否利用大量示例
Long instruction	约束是否在长上下文中保持

11. Coding Eval：从 HumanEval 到 Repo-Level / Economic Tasks

代码评测的演进很典型：单函数生成 → repo-level issue resolution → 真实 freelance 工作。SWE-bench 将任务定义为真实 GitHub issue 与代码库修复；SWE-bench Verified 再由人工验证 500 个更可靠样本；SWE-Lancer 进一步使用真实 Upwork 软件工程任务与端到端测试。[16]

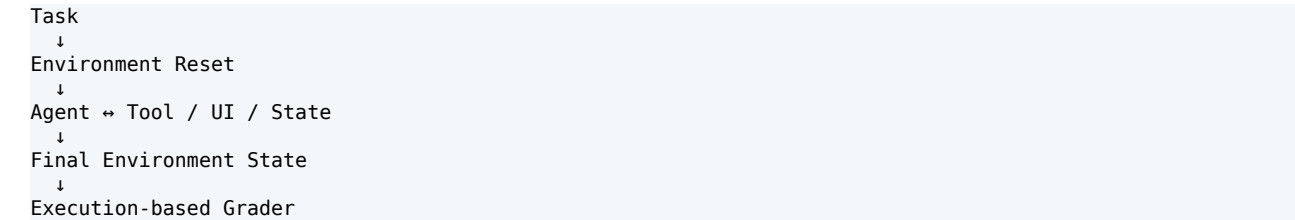
层级	代表任务	主要 grader	隐藏问题
Function	代码补全/函数生成	unit tests	上下文短、容易污染
Repository	SWE-bench	repo tests / patch	环境、依赖、issue interpretation
Economic	SWE-Lancer	end-to-end tests / manager choice	成本、真实 workflow、长时程

11.1 Coding Eval 必须记录的系统变量

- Agent scaffold、tool list、shell/network 权限。
- 最大 steps、tokens、wall-clock、parallelism。
- Docker/environment version、dependency cache。
- test pass、patch size、regression tests、cost。

12. Agent Eval：最终状态比漂亮轨迹更重要

WebArena 用可复现网站环境测试长程 web task；OSWorld 则把评测扩展到真实桌面操作系统和多应用任务，并为每个任务提供状态初始化与 execution-based evaluation script。[4][5]



Agent Eval 第一原则

不要只判断 final answer 文本。真正的 Agent 成功通常是“环境状态被正确改变”：文件创建、网页提交、数据库记录、代码测试通过等。

13. Agent Trajectory Eval：Outcome + Process + Cost

指标层	例子	用途
Outcome	task success、tests pass	最终有没有完成
Process	invalid tool calls、retries、loops	为什么成功/失败
Efficiency	steps、tokens、latency、API/tool cost	同能力下谁更经济
Robustness	tool error recovery、state drift	真实部署可靠性
Safety/Control	权限越界、危险动作、确认机制	高影响任务 gating

Anthropic 的 agent eval 工程总结建议根据任务组合 grader：例如 research agent 可分别检查 groundedness、coverage、source quality，而 coding agent 更多依赖 execution-based tests。[13]

13.1 Trace Grading

2025 年之后，Agent eval 越来越重视 trace-level diagnosis：即使最终 task success 相同，也要区分“合理路径成功”和“偶然/投机成功”。OpenAI AgentKit 的 eval 能力也明确加入 trace grading。[18]

14. Test-Time Compute：公平比较必须报告预算

现代 reasoning/agent 系统往往通过更多采样、更长思考、搜索或 tool calls 换取性能。Eval 必须把能力曲线表达为 performance vs compute，而不是只报告一个点。

预算变量	例子
Samples	1 / 4 / 16 / 64 candidates
Reasoning tokens	low / medium / high budget
Tool calls	max 10 / 50 / 200
Search nodes	beam / MCTS / best-of-N
Wall-clock	5min / 30min / 2h
Dollar cost	per task / per solved task

Utility = TaskSuccess - λ1*TokenCost - λ2*ToolCost - λ3*Latency

15. Statistical Reliability：0.3 分提升可能只是噪声

15.1 必须报告的不确定性

- 样本比例指标：binomial confidence interval（如 Wilson interval）。
- 开放式平均分：bootstrap confidence interval。
- pairwise win rate：bootstrap / paired test。
- Agent stochasticity：多 seed / 多 rollout 重复。

15.2 Paired Evaluation 优先于独立平均

比较 Model A/B 时，如果对同一批样本进行 paired evaluation，可以利用样本级差异降低方差。尤其在昂贵 Agent eval 中，paired bootstrap 往往比只比较两个总体均值更有信息。

15.3 Multiple Comparisons

大量 benchmark/slice 同时比较时，很容易偶然“显著”。发布决策应提前定义 primary metrics、gates 与必要的多重检验修正，而不是训练后挑最好看的数字。

16. Slicing：平均分是最危险的指标之一

Slice	为什么重要
Difficulty	总体提升可能只来自 easy tasks
Domain	某领域显著退化
Language	非英语能力被平均掩盖
Context length	短文本提升、长文本退化
Prompt type	constraint / tool / reasoning
Risk severity	低风险平均分掩盖高风险 failure
User cohort	真实产品分布不均
Model confidence	高置信错误比普通错误更危险

Know-How

Eval dashboard 的基本单位应该是 “metric × slice × model version”，而不是单个总榜。

17. Worst-Case / Tail Reliability：平均成功率不等于可部署

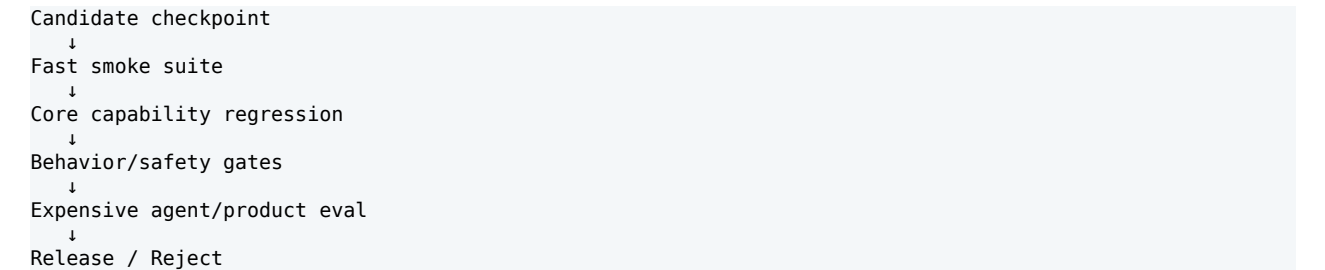
很多产品关心的是 P95/P99 failure、严重错误率或连续任务稳定性，而不是平均准确率。HealthBench 也强调 worst-case reliability 与在欠明确问题下主动寻求上下文的重要性。[19]

指标	说明
Critical Failure Rate	高严重度错误比例
Worst-slice Accuracy	最差 domain/语言/难度切片
CVaR-style metric	关注最差一部分样本
Consecutive Success	连续 N 次任务都成功的概率
Escalation Quality	不确定时能否正确寻求人类/更多上下文

17.1 Long-horizon reliability 的乘法效应

如果每一步独立成功率是 p ，一个需要 N 个关键步骤的任务，其粗略端到端成功率可能近似 p^N 。即使单步 98% 很高，长程系统仍可能表现不稳定。这也是 Agent eval 必须走向端到端环境测试的原因之一。

18. Regression Eval：新能力不能靠破坏旧能力换来



层	特点	频率
Smoke	小、快、确定	每 checkpoint
Core Regression	核心能力私有集	每日/主要实验
Full Benchmark	广覆盖	里程碑
Agent/Product	昂贵、真实	候选发布
Red Team / Audit	极端和对抗	重大版本

19. Eval Leakage：不要用同一套题训练、调参、选模型、发榜

最常见的 Eval 失真并不是直接把答案放进训练集，而是反复围绕公开 benchmark 调 prompt、采样参数、数据配方与 checkpoint。即使没有“训练数据泄漏”，这仍然是一种 benchmark overfitting。

19.1 推荐的数据隔离

Train set
Development Eval → 高频调试
Private Regression → 模型选择/版本对比
Release Eval → 少量访问
External/Live Eval → 最终泛化检查

原则

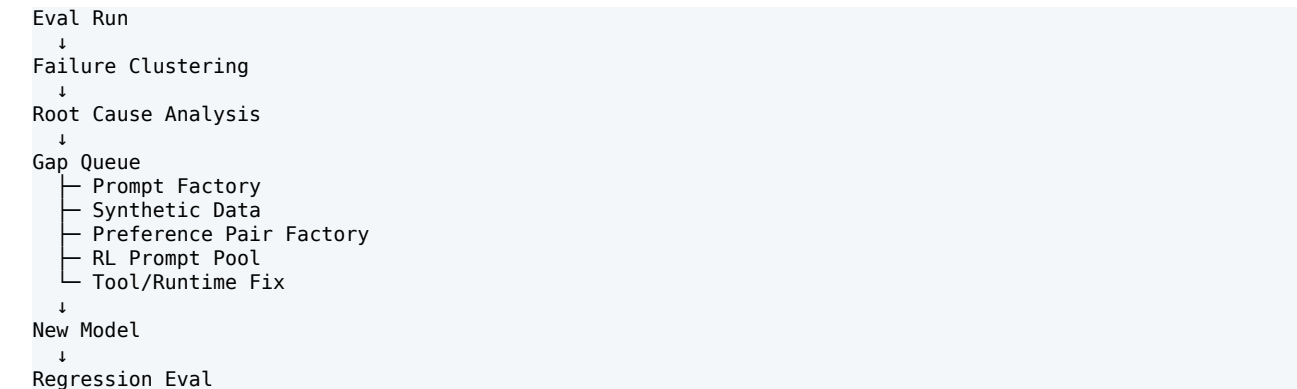
Eval set 也需要 access control、版本、访问日志与 provenance。把 benchmark 当成生产资产治理，而不是一堆 JSON。

20. Failure Taxonomy：Eval 的价值在于制造“可训练失败”

一个 71.2% 的分数几乎不能指导下一轮训练。真正有价值的是把 28.8% failure 分成能映射到 Data/Reward/Tool/Runtime 的类型。

Failure 类别	例子	典型后训练动作
Knowledge Gap	缺事实/领域知识	CPT/SFT/RAG data
Reasoning Gap	路径错误、算术/逻辑失败	reasoning data / RLVR
Instruction Gap	漏约束、格式错	SFT / preference
Tool Selection	选错工具	tool SFT / agent RL
Tool Argument	参数错误	schema/tool-call training
Recovery Failure	失败后不重试/乱试	trajectory SFT / RL
Verifier Gaming	钻 grader 漏洞	reward redesign / hidden tests
Overconfidence	错误但高置信	calibration / uncertainty training
Cost Pathology	能做但过度消耗	cost-aware reward / runtime policy

21. Eval → Failure Mining → Training：真正的闭环



这一步把 Evaluation 从“排行榜工具”变成训练控制器：每一类失败都必须有 owner、severity、frequency、root cause 与 remediation route。

21.1 Gap Priority

$$\text{Priority(gap)} \propto \text{Frequency} \times \text{Severity} \times \text{ProductImpact} \times \text{Learnability} / \text{FixCost}$$

不要只训练最常见 failure，也不要只训练最难 failure；应优先选择“影响大且当前模型可学习”的缺口。

22. Public Benchmark vs Private Eval vs Online Metrics

层	作用	优点	风险
Public Benchmark	外部可比性	生态标准	污染/过拟合
Private Eval	内部真实目标	难被针对	复现性低
Online Shadow/A-B	真实用户效用	最接近产品	噪声、成本、伦理/实验治理
Human Review	高质量开放评价	可处理复杂产物	贵、慢、一致性

最佳实践

三层缺一不可：Public benchmark 用于定位生态位置；Private eval 用于模型开发与 release gating；Online/product metrics 用于验证真实效用。

23. 真实世界 Eval：GDPval / PaperBench / SWE-Lancer 的共同方向

2025 年的一批评测明显从短答案 benchmark 转向“真实工作产物”：GDPval 覆盖 44 个职业的经济价值任务；PaperBench 测试端到端论文复现；SWE-Lancer 使用真实 freelance 软件工程任务。共同趋势是：任务更长、产物更复杂、rubric 更细、评价更接近真实工作流。[14][15][16]

Benchmark	评测单位	Grading 思路	启示
GDPval	职业工作产物	expert preference / rubric	测真实经济任务而非学术短题
PaperBench	论文复现项目	hierarchical rubric + judge benchmark	复杂任务必须拆成可评分子目标
SWE-Lancer	真实工程交付	end-to-end tests / manager decisions	把能力映射到经济价值与任务成本

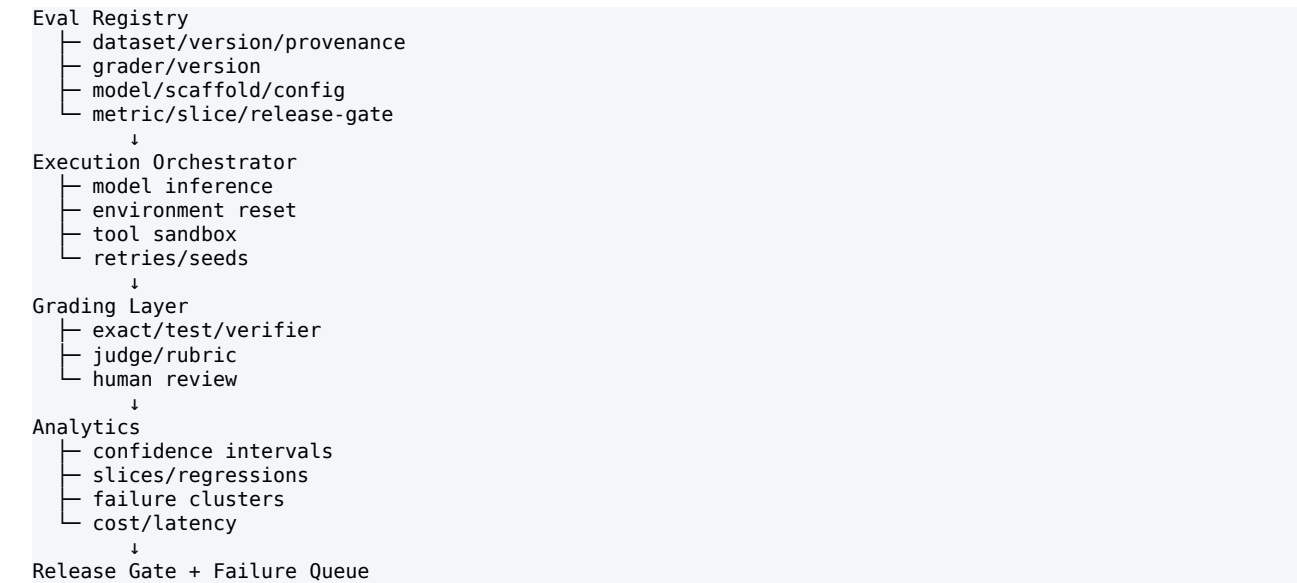
24. Behavioral Evaluation：从固定题库到自动场景生成

Anthropic Bloom 将研究者指定行为转化为自动生成的评测场景，并量化行为发生的频率与严重程度。这代表 Evaluation 的另一条演进：不只动态更新“能力题”，也动态生成 behavioral probes，以应对固定行为 benchmark 被训练集污染或失去区分力。[17]

24.1 Behavioral Eval Contract

- Behavior definition：明确、可观测、含边界。
- Scenario generator：覆盖多上下文、多诱因。
- Grader：行为发生与严重度。
- Calibration：与人工标签的一致性。
- Adversarial variants：防止模型只记住模板。

25. Eval System Architecture：生产级服务应长什么样？



25.1 每次 Eval Run 必须可复现

字段	示例
model_checkpoint	model-v42-sha
system_prompt	prompt-v18
scaffold	agent-runtime-v7
dataset_version	swe-private-2026-08-01
grader_version	test-suite-v9 / judge-v6
sampling	T=0.6, top_p=.95, max=32k
budget	16 rollouts / 100 tool calls
environment	docker image hash
seed	per task seeds
timestamp	for live/dynamic evals

26. Evaluation Contract：可直接复用的 YAML

```
eval_suite: coding_agent_release_v12
objective: "reliably resolve real repository issues"
unit_of_eval: task_trajectory
model:
  checkpoint: candidate
  scaffold: agent_runtime_v7
datasets:
  - name: private_repo_regression
    weight: 0.45
  - name: swe_bench_verified
    weight: 0.25
  - name: fresh_failure_mining
    weight: 0.30
graders:
  outcome: hidden_tests
  process: trace_rules
metrics:
  - task_success
  - pass_at_1
  - invalid_tool_rate
  - cost_per_solved_task
slices:
  - difficulty
  - language
  - repo_size
  - failure_type
release_gates:
  task_success_delta: ">= +2.0pp"
  critical_regression: "none"
  cost_increase: "<= 10%"
statistics:
```

```
bootstrap_ci: 0.95  
paired_comparison: true
```


27. 18 类常见 Evaluation Failure Modes

#	Failure Mode	根因/修复
1	Benchmark-only optimization	缺真实产品任务 → 加 private/product eval
2	Benchmark contamination	公开数据泄漏 → temporal/private/hidden sets
3	Judge is ground truth	judge 未校准 → objective verifier/human audit
4	One-number leaderboard	平均分掩盖 trade-off → multi-metric + slices
5	No compute budget control	pass@k/agent search 不公平 → 固定并报告预算
6	No stochastic repeats	agent 结果方差大 → multi-seed / CI
7	Static saturated set	模型接近满分 → refresh/harder/live eval
8	Weak rubric	开放任务总分随意 → proposition-level rubric
9	Leaky dev/release eval	反复调参 → 分层 holdout/access control
10	Outcome-only diagnosis	知道失败但不知道原因 → trace/process metrics
11	Process-only reward	轨迹漂亮但没完成 → outcome verifier 优先
12	Environment drift	依赖/网页变化 → pinned env / reset validation
13	Flaky tests	grader 本身不稳定 → grader unit tests
14	No severity weighting	小错与严重错误等价 → severity-aware metrics
15	No cost accounting	性能靠极大预算堆出 → cost-performance frontier
16	No failure ownership	分析后不进入训练 → gap queue + owner
17	Eval overfitting	针对私有题反复硬编码 → refresh + external live checks
18	Model-system confusion	换 scaffold 被说成模型提升 → component ablations

28. Evaluation Maturity : L0 → L4

等级	特征	核心跃迁
L0 Benchmark	手工跑几个公开榜	从感觉到基本量化
L1 Regression Suite	版本化私有集 + 自动 grader	从一次性到持续回归
L2 Portfolio	多能力/行为/成本切片	从总分到多目标
L3 Interactive System Eval	环境、trace、统计可靠性	从文本到系统/Agent
L4 Adaptive Eval Controller	动态任务生成 + failure mining + release gates	从测量工具到训练控制系统

29. Know-Why / Know-How 总表

模块	Know-Why	Know-How
Objective Contract	不定义“好”就无法证明进步	目标编译成 task/grader/metric/slice
Eval Portfolio	单榜无法覆盖真实能力	public + private + live + product
Grader	评分器决定你看到的世界	优先 verifier；Judge 必须 calibration
Statistics	小提升可能是噪声	paired tests、bootstrap、CI、多 seed
Slicing	平均分隐藏长尾失败	difficulty/domain/language/risk slicing
Agent Eval	文本不是最终任务状态	execution-based final-state grading
Contamination	公开 benchmark 会被训练吸收	temporal/private/hidden/refresh
Cost Eval	测试时算力也是能力变量	performance-vs-compute frontier
Regression	局部提升会破坏其他能力	release gates + critical slices
Failure Mining	分数本身不能训练模型	cluster → root cause → gap queue

30. 最终心智模型：Evaluation 是后训练的传感器与控制器

核心公式

Effective Eval ≈ Coverage × Grader Validity × Statistical Reliability × Distribution Relevance × Diagnostic Value / (Execution Cost + Maintenance Cost + Leakage Risk)。

一个成熟 Evaluation 系统的目标不是“拥有更多 benchmark”，而是持续回答五个问题：① 当前模型真正会什么？② 哪些能力/行为正在退化？③ 哪些失败最影响真实用户？④ 这个变化是否统计可信、是否来自更多 test-time compute？⑤ 这些失败怎样被转成下一轮 Data、Preference、Reward、Tool 或 Runtime 改进？

Objective → Eval Contract → Run → Grade → Slice → Failure Taxonomy → Gap Queue → Post-Training → Regression

因此，Evaluation 的最终产物不是排行榜，而是一个可执行的“学习方向”。这也是为什么 Eval 应当从项目末尾前移到后训练系统的最上游。

参考资料与推荐阅读顺序

- [1] HELM: Holistic Evaluation of Language Models — <https://arxiv.org/abs/2211.09110>
- [2] LiveBench: A Challenging, Contamination-Free LLM Benchmark — <https://arxiv.org/abs/2406.19314>
- [3] HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly — <https://arxiv.org/abs/2410.02694>
- [4] WebArena: A Realistic Web Environment for Building Autonomous Agents — <https://arxiv.org/abs/2307.13854>
- [5] OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments — <https://arxiv.org/abs/2404.07972>
- [6] AgentBench: Evaluating LLMs as Agents — <https://arxiv.org/abs/2308.03688>
- [7] SWE-bench: Can Language Models Resolve Real-World GitHub Issues? — <https://arxiv.org/abs/2310.06770>
- [8] SWE-bench Verified (OpenAI) — <https://evals.openai.com/swe-bench-verified>
- [9] Benchmark Data Contamination of Large Language Models: A Survey — <https://arxiv.org/abs/2406.04244>
- [10] JudgeBench: A Benchmark for Evaluating LLM-based Judges — <https://arxiv.org/abs/2410.12784>
- [11] OpenAI Evals — <https://evals.openai.com/>
- [12] OpenAI: How evals drive the next chapter in AI for businesses — <https://openai.com/index/evals-drive-next-chapter-of-ai/>
- [13] Anthropic: Demystifying evals for AI agents — <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- [14] PaperBench — <https://evals.openai.com/paperbench>
- [15] GDPval — <https://evals.openai.com/gdpval>
- [16] SWE-Lancer — <https://evals.openai.com/swe-lancer>
- [17] Anthropic Bloom automated behavioral evaluations — <https://www.anthropic.com/research/bloom>
- [18] OpenAI AgentKit - Evals and trace grading — <https://openai.com/index/introducing-agentkit/>
- [19] OpenAI HealthBench — <https://openai.com/index/healthbench/>

推荐阅读顺序

第一层：HELM → LiveBench，建立“多维评测 + contamination/refresh”基础。第二层：SWE-bench / WebArena / OSWorld，理解从静态文本到交互式环境的转变。第三层：JudgeBench + Anthropic Agent Evals，理解 grader 与 agent evaluation 的可靠性问题。第四层：PaperBench / GDPval / SWE-Lancer / Bloom，理解 2025-2026 年真实工作任务、复杂 rubric 与动态 behavioral eval 的前沿方向。